

A.X K1 Technical Report

SK Telecom



<https://huggingface.co/skt/A.X-K1>

Abstract

We introduce A.X K1, a 519B-parameter Mixture-of-Experts (MoE) language model trained from scratch. Our design leverages scaling laws to optimize training configurations and vocabulary size under fixed computational budgets. A.X K1 is pre-trained on a corpus of approximately 10T tokens, curated by a multi-stage data processing pipeline. Designed to bridge the gap between reasoning capability and inference efficiency, A.X K1 supports explicitly controllable reasoning to facilitate scalable deployment across diverse real-world scenarios. We propose a simple yet effective Think-Fusion training recipe, enabling user-controlled switching between thinking and non-thinking modes within a single unified model. Extensive evaluations demonstrate that A.X K1 achieves performance competitive with leading open-source models, while establishing a distinctive advantage in Korean-language benchmarks.

1 Introduction

Large language models (LLMs) have significantly progressed in real-world applications such as chatbots (Achiam et al., 2023; Comanici et al., 2025), search engines (Perplexity AI; Xiong et al., 2024), and coding assistants (Guo et al., 2024; Hui et al., 2024). In parallel, LLMs have rapidly advanced in reasoning capability through scaling model parameters and reinforcement learning (Kimi Team, 2025; DeepSeek-AI, 2025; Yang et al., 2025). These advances are largely driven by increased model capacity (Comanici et al., 2025; DeepSeek-AI, 2025) and expanded test-time computation (Snell et al., 2024; Muennighoff et al., 2025). However, translating such capabilities into practical deployments introduces several fundamental challenges.

First, achieving strong reasoning performance under practical inference constraints requires carefully balancing massive knowledge capacity with deployment feasibility. To address this challenge, we design the architecture of our large-scale language model by referencing recent scaling laws for Mixture-of-Experts (MoE) models (Tian et al., 2025) and vocabulary size (Tao et al., 2024). In particular, the model is configured as a 519B-parameter MoE architecture with 33B active parameters, pushing model capacity within practical hardware constraints, and adopts an optimized vocabulary size of 160K. This architectural choice prioritizes inference throughput and knowledge density over strictly compute-optimal training efficiency, reflecting the requirements of real-world, large-scale deployment.

Second, as LLMs are increasingly deployed in real-world systems, an efficiency challenge arises: the reasoning process relies on extensive token generation, causing prohibitive latency and inference costs due to the thinking process (Yang et al., 2025; DeepSeek-AI, 2025). While the thinking process demonstrates superior performance on complex tasks, applying deep reasoning to simple queries results in unnecessary computational waste and degraded user experience due to slow response times. Thus, practical applications necessitate *flexible control over the extent of reasoning*, allowing systems to balance performance with computational efficiency rather than indiscriminately maximizing reasoning depth. To address this, we introduce a simple yet effective *Think-Fusion* training recipe that enables A.X K1 to support both explicit thinking and standard (direct) modes within a single unified model. This capability empowers users to dynamically adjust the computational cost based on task complexity, bridging the gap between high-capability reasoning and scalable real-world deployment.

In this work, we present **A.X K1**, a *compute-efficient Mixture-of-Experts (MoE) language model trained from scratch* to address the above challenges. Grounded in MoE and vocabulary scaling laws (Tian et al., 2025; Tao et al., 2024), A.X K1 is configured with 519B total parameters and 33B active parameters, enabling high-capacity modeling under a fixed computational budget. Beyond architectural scale, A.X K1 explicitly supports both reasoning-intensive and direct inference modes through a unified training framework, allowing users to control computational cost at inference time. As a result, the model achieves competitive performance among leading open-weight peers with comparable active parameter sizes across a broad range of benchmarks, including knowledge, instruction following, mathematics, and coding.

Beyond its technical contributions, A.X K1 represents a strategic effort to establish a robust *Sovereign AI* ecosystem. By developing a high-performance foundation model that deeply understands the linguistic and cultural characteristics of Korea, we aim to reduce reliance on foreign proprietary models and

accelerate the adoption of trustworthy AI across local industries, government, and academia. This initiative aligns with our broader objective of democratizing advanced reasoning capabilities and tailoring them to the specific needs of the Korean AI ecosystem.

We highlight three key aspects of A.X K1 as follows:

- (1) **Compute-Efficient Pre-training:** We demonstrate large-scale MoE pre-training with 519B total parameters using approximately 10T tokens, guided by scaling laws. Despite a fixed compute budget, the model approaches the performance of larger-scale systems trained with substantially more data, indicating strong computational efficiency and further headroom with additional resources.
- (2) **Flexible Reasoning via Think-Fusion:** We introduce a methodology for constructing a practical unified model employing our *Think-Fusion* recipe. By synergizing *Model Merging* and *Think-Fusion SFT*, we efficiently unify hybrid capabilities within a single architecture. The resulting model supports a *non-thinking mode* for concise, low-latency responses and a *thinking mode* for extensive reasoning chains, empowering users to flexibly optimize resource allocation based on task complexity.
- (3) **Frontier-Scale Engineering Capability:** A.X K1 demonstrates the feasibility of end-to-end engineering for 500B+ parameter models, including stable training, reproducible pipelines, and system-level optimizations. These efforts establish a foundation for scaling toward frontier-class model development.

Extensive evaluations show that A.X K1 matches or exceeds state-of-the-art open-source baselines across English and Korean benchmarks, while providing explicit, user-controllable switching between thinking and non-thinking modes at inference time. Beyond benchmarks, our results validate the feasibility of unifying implicit and explicit reasoning processes within a single parameter space. We release A.X K1¹, positioning it as a practical, controllable foundation model.

Organization. The remainder of this report details the model architecture and tokenizer design (Sec. 2), pre-training (Sec. 3), post-training (Sec. 4), and evaluation results (Sec. 5).

2 Architecture

Table 1: Architecture Overview of A.X K1 MoE Model, which adopts Multi-head Latent Attention (MLA).

Models	Total Params.	Activated Params.	Layers	Heads (Q / KV)	Routed / Activated Experts	Shared Experts	Context Length
A.X K1	519B	33B	61	64	192 / 8	1	128K

2.1 Resource-Constrained Model Design

A.X K1 was developed over approximately four months as part of the ‘Sovereign AI foundation model’ project² funded by the Korean government. Within this constrained timeline, our objective was to deliver a 519B-parameter MoE model that remains competitive with contemporary frontier models. Under a fixed computational budget, we carefully budgeted the available training FLOPs and used these estimates to choose a principled trade-off between model scale and the number of training tokens. These constraints shaped several efficiency-driven design decisions—from the MoE configuration to the training recipe—so that we could maximize capability within a feasible time and compute. While the resulting model inevitably reflects the limits of the available budget, our work demonstrates a practical and reproducible path to building a large-scale MoE model under real-world constraints.

2.2 Model Configuration

The A.X K1 model is a large-scale Mixture-of-Experts (MoE) language model with a total of 519B parameters and 33B active parameters, a configuration strategically chosen to maximize knowledge capacity within our hardware constraints while ensuring feasible inference latency. Detailed architectural specifications for A.X K1 are presented in Table 1. For the pre-training compute budget, given a strict training timeline of approximately 75 days on 1,024 H200 GPUs, and assuming a sustained training

¹<https://huggingface.co/skt/A.X-K1>

²The Ministry of Science and ICT(MSIT), Republic of Korea, through the National IT Industry Promotion Agency(NIPA) (Grant No. PJT-25-080042).

throughput of 3.84×10^{14} FLOPs/s/GPU (estimated based on efficiency characteristics in (Liu et al., 2024)), the total compute budget was derived as $C \approx 2.548 \times 10^{24}$ FLOPs. Under this fixed architecture and compute budget, subsequent architectural trade-offs were guided by established scaling principles for MoE models.

Regarding the granularity of expert capacity, we referenced the scaling laws in (Tian et al., 2025) to balance efficiency with engineering stability. Granularity is defined as $(2 \times d_{\text{model}})/d_{\text{expert}}$, where d_{model} denotes the FFN hidden size and d_{expert} the intermediate size of each expert. While a granularity range of approximately 8–12 is reported to be optimal under well-balanced expert routing, this condition is not always achieved in practice. Given $d_{\text{model}} = 7,168$, we selected $d_{\text{expert}} = 2,048$, resulting in a granularity ($G = 7$) slightly below the nominal optimal range. This choice aligns with the observation in (Tian et al., 2025) that under imperfect expert load balancing, moderately lower granularity values can yield better efficiency and training stability, favoring robustness over idealized balanced assumptions.

The architectural design of A.X K1 adopts several widely used components that have proven effective in large-scale MoE systems (Liu et al., 2024; Yang et al., 2025; GLM-4.5 Team, 2025). These include Multi-head Latent Attention (MLA) (Liu et al., 2024), which improves Key–Value cache efficiency and reduces memory overhead under long-context settings. Auxiliary-loss-free load balancing is adopted for expert routing, and multi-token prediction (MTP; Gloeckle et al., 2024) is used to support speculative decoding during inference (Leviathan et al., 2023).

To improve training stability, we adopt a dual normalization scheme inspired by the design in (Gemma Team, 2025), applying RMSNorm (Zhang & Sennrich, 2019) both before and after the MLP and MoE layers. This configuration effectively reduced loss spikes observed during the early stages of training, as shown in Fig. 1.

Early GLM-4 model series³ closely follow the Gemma architecture, employing post-self-attention normalization, followed by pre-MLP and post-MLP normalizations, resulting in stacked normalization layers. In contrast, we streamlined this design by applying RMSNorm strictly at the input and output of the MLP/MoE blocks. This simplified dual-normalization approach maintains training stability while reducing architectural redundancy compared to earlier implementations.

Separately, informed by empirical observations in (Kimi Team, 2025), we configured the model with 64 attention heads and integrated shared dense experts as an operating point that prioritizes inference efficiency by reducing attention overhead while enhancing knowledge sharing. We also evaluated alternating sliding-window attention (OpenAI, 2025). While small-scale experiments showed promising gains, the method exhibited unreliable behavior with large-scale context parallelism in our pre-training dry runs. Prioritizing training stability for long-context pre-training, we excluded this mechanism from the final architecture.

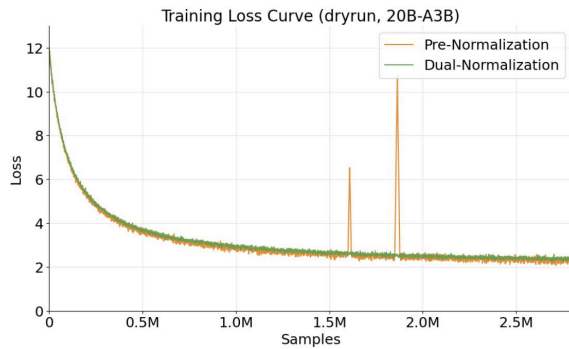


Figure 1: Training loss curves for Pre-Normalization and Dual-Normalization during early training of A.X K1 Light (20B-A3B, small-scale model for dry-run).

2.3 Tokenizer

We developed a tokenizer designed to efficiently process a diverse training corpus comprising multi-lingual text, reasoning-oriented data, and code. Employing the standard Byte-level Byte-Pair Encoding (BBPE) algorithm (Sennrich et al., 2016), we selected a vocabulary size of 160K to ensure comprehensive coverage across English, Korean, Chinese, Japanese, and Spanish. While recent studies propose Morpheme-aware BBPE (Asgari et al., 2025), we opted for the standard BBPE formulation for this project. A detailed analysis of tokenization efficiency is presented in Section 5.4.

Design of Vocabulary Size. We optimize the vocabulary size based on the *Derivative-based Estimation* from (Tao et al., 2024), which suggests a theoretical baseline of approximately 132,500 for our architecture ($N_{nv} = 32B$, $d = 7168$, indicating non-vocabulary parameters and embedding dimension respectively) under compute-optimal assumptions Hoffmann et al. (2022). However, to accommodate our 10T-token training regime, we increased the optimal size from 132,500 to 163,840 (approximately a 25% increase), based on preliminary empirical observations from smaller-scale models. This consideration motivates

³https://docs.vllm.ai/en/stable/api/vllm/model_executor/models/glm4

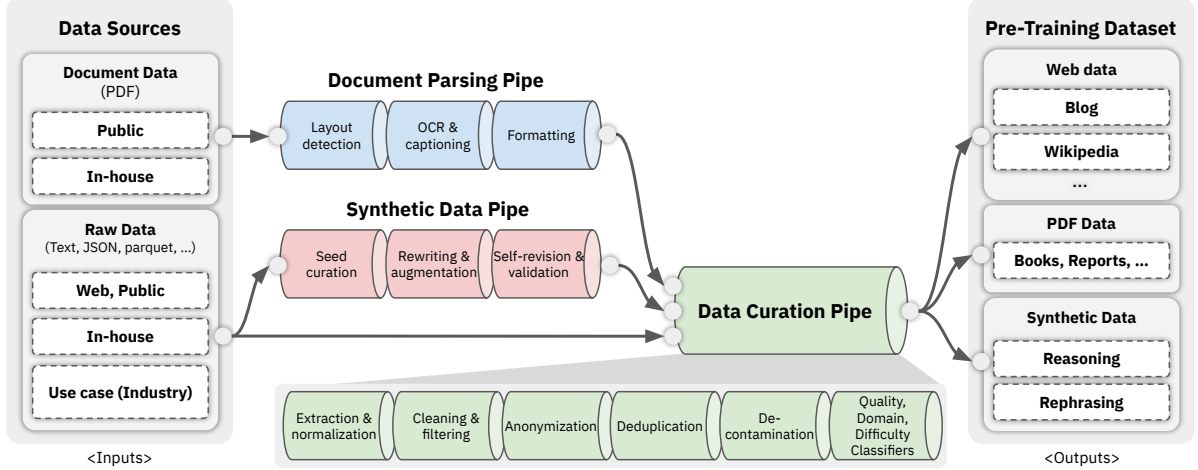


Figure 2: **A Data Processing Framework for Constructing High-Quality Pre-training Datasets.** The diagram illustrates the transformation of raw data sources (documents and raw data) into a final training corpus through document parsing, synthetic data generation, and a multi-stage curation pipeline (including cleaning, anonymization, deduplication, and diverse classifiers).

our decision to expand the vocabulary size relative to the 102,400-token vocabulary used in our previous model (SKT, 2025a).

This adjustment balances theoretical scaling with practical constraints. First, it accounts for the upward shift in optimal vocabulary size required by data-rich regimes (Tao et al., 2024) while avoiding the excessive inference latency associated with overly large vocabularies. Second, the selected size is a multiple of 128, ensuring hardware alignment for efficient Tensor Core utilization during embedding and projection operations. This choice represents a robust trade-off between scaling laws and hardware efficiency.

3 Pre-training

3.1 Pre-training Dataset

We constructed a comprehensive pre-training corpus comprising 10 trillion tokens using a data pipeline shown in Fig. 2. This dataset encompasses diverse web corpora alongside high-quality content spanning various domains, including code, STEM (Science, Technology, Engineering, and Mathematics), reasoning, books, and synthetic data. All data sources are refined through the data processing pipeline that utilizes multi-stage quality filtering and fine-grained classifiers for attributes such as domain and difficulty.

To further enrich the corpus, we generated synthetic data using a lightweight language model (SKT, 2025b) by rephrasing scraped Korean web data and designing diverse prompting strategies, including Wikipedia and QA. Additionally, we employed a finetuned VL model (SKT, 2025c) to extract text from a massive collection of PDF documents. The extracted text was subsequently refined through a PDF processing pipeline to ensure high quality. Collectively, these augmentation strategies contributed trillions of additional high-quality tokens to the final dataset. We detail these processes in the following section and provide the mixture ratio for each training stage in Appendix B.1.

Document parsing pipeline. PDF documents, ranging from academic literature and books to technical reports, contain a wealth of high-quality knowledge but often suffer from structural complexity. To unlock this data, we established a robust document processing pipeline. We use an in-house vision-language model designed to interpret complex visual content specific to Korean. We process this with finetuned layout detection and parsing models, trained in-house to precisely identify and reconstruct diverse document elements (e.g., headers, multi-column text, tables). This pipeline was applied to a massive collection of crawled web documents, internal repositories, and books. By filtering parsing errors and preserving logical structures, we successfully extracted a large-scale, high-quality corpus from sources.

Synthetic data pipeline. We implement a dual synthetic data pipeline designed to enhance both reasoning depth and knowledge breadth. Our generation strategy is categorized into two complementary domains:

- **Seed corpus-based Synthetic Data.** We focus on transforming existing high-quality text into complex training signals. We first curate a seed corpus comprising *high-difficulty documents from specific sources* (e.g., STEM) and intricate logical texts. Utilizing an ensemble of open-source LLMs, including in-house models (SKT, 2025a), we restructure this content into multi-step reasoning chains or agentic workflows. This process effectively converts static information into dynamic, process-oriented data, significantly boosting the model’s capabilities in complex problem-solving and agentic planning.
- **Topic-based Synthetic Data.** To systematically elicit the latent *parametric knowledge* stored within teacher models, we employ a topic-driven generation strategy. We aggregate a diverse set of prompts derived from real-world user queries and extract core technical keywords from STEM textbooks. These topics serve as generation seeds, prompting LLMs to synthesize comprehensive explanations and new knowledge contexts that might be sparse in the pre-training corpora.

All generated datasets are subsequently processed through the suite of classifiers detailed in our data curation pipeline.

Data curation pipeline. The quality and distribution of pre-training data are crucial determinants of large language model performance. To ensure our model acquires robust reasoning capabilities and broad world knowledge, we implemented a sophisticated three-stage data curation pipeline: **Quality Filtering**, **Domain Classification**, and **Difficulty Scoring**. We finetune our lightweight Korean text encoder (SKT, 2025d) designed for processing long-context sequences, to train all classifiers.

- **High-Quality Filtering.** To mitigate the impact of noisy and low-information content inherent in web-scale corpora, we employ a multiple filtering process. This process combines heuristic filtering, deduplication, and model-based filtering. This ensures that the base corpus maintains high linguistic coherence, educational value, and information density.
- **Domain-Aware Mixture and Upsampling.** Diversity in training data is essential, but uniform sampling often leads to the under-representation of reasoning-intensive domains. We developed a *Domain Classifier* trained on a taxonomy of academic and general topics. Based on the classifier’s output, we construct a weighted data mixture that strategically upsamples reasoning-related domains. In particular, we heavily weigh STEM. This re-balancing strategy is designed to enhance the model’s logical reasoning and problem-solving skills.
- **Difficulty Scoring for Curriculum Learning.** Beyond domain classification, optimizing the order of data presentation is critical. We introduced a *Difficulty Classifier* that scores documents based on lexical complexity, syntactic structure, and semantic depth. Leveraging these scores, we adopt a curriculum Learning (Bengio et al., 2009). The training process initiates with simpler, fundamental concepts and progressively introduces higher-difficulty samples. This scheduling facilitates better generalization on complex reasoning tasks.

3.2 Pre-training Process

To effectively optimize A.X K1 across varying data distributions, we adopt the Warmup-Stable-Decay (WSD) scheduler (Hu et al., 2024) within our multi-stage pre-training pipeline. In our setup, the stages of the WSD scheduler do not directly correspond to the pre-training stages defined by data composition. Instead, the pre-training of A.X K1 is organized into three stages: general knowledge training, advanced knowledge and reasoning training, and long-context extension, based on training objectives and data characteristics. The following sections describe each stage in detail.

1. *General Knowledge Stage (Stable Phase):* In the initial pre-training stage, A.X K1 models are trained on a corpus of 7 trillion tokens with a sequence length of 4,096. This phase establishes a robust foundation in both linguistic proficiency and general world knowledge. Aligning with the WSD strategy, we maintain a constant learning rate throughout this stage to maximize knowledge acquisition.
2. *High-Quality Reasoning Stage (Decay Phase):* Following the initial stage, we transition to a specialized phase focused on enhancing reasoning capabilities. This stage utilizes a curated corpus of 1.66 trillion tokens, strictly filtered for high-complexity web data. Furthermore, we incorporate a diverse mixture of domain-specific datasets, including STEM, reasoning-intensive tasks, refined PDF content, and high-quality synthetic data. In this stage, we initiate the learning rate decay to stabilize the model on high-quality distributions.
3. *Long-Context Adaptation:* To enable long-context capabilities, we adopt a multi-stage curriculum that progressively extends the model’s context length from 4K to 16K and subsequently to 32K, using a total of 600 billion tokens. This design is motivated by the demands of agentic tasks,

which require long-horizon reasoning, planning, and action over extended contexts. Accordingly, the long-context training stages incorporate data featuring long chains of thought and repository-level code. We maintain a constant learning rate throughout all extension phases.

3.3 Hyperparameters

Learning rate and batch size were selected based on the MoE scaling laws proposed by (Tian et al., 2025), which were introduced earlier in the context of model size and architecture design. Under these scaling laws, the maximum learning rate is given by $1.1576 \times C^{-0.1529}$, which evaluates to approximately 2.147×10^{-4} for the estimated compute budget; in practice, the peak learning rate was set to 2.1×10^{-4} . The scaling law also recommends a global batch size of $0.0694 \times C^{0.3644}$, corresponding to roughly 54 million tokens.

Since larger batch sizes are generally beneficial for throughput in MoE training, the effective batch size was increased to 64M tokens. During training, the GPU count was later expanded to 1,536, at which point the global batch size no longer aligned with a power-of-two configuration; to maintain high utilization under the updated hardware configuration, the batch size was further adjusted to approximately 75M tokens.

To mitigate gradient spikes and ensure stability during the early training phase, a global batch size ramping-up strategy was applied, linearly increasing the batch size from 2,048 to 16,384 in increments of 1,024 during the warmup period, which spans 72 million samples (approximately 295B tokens with a sequence length of 4,096).

We adopt the AdamW optimizer for pre-training, with momentum parameters set to $\beta_1 = 0.9$ and $\beta_2 = 0.95$. Although alternative optimizers such as Muon (Liu et al., 2025) were evaluated, they were not adopted in this iteration due to the observed degradation in training throughput in our framework. With recent improvements addressing these performance issues, we plan to adopt Muon or its variants in subsequent versions.

In addition to the general optimization hyperparameters, several MoE-specific settings were configured to balance routing stability and expert utilization. The auxiliary loss coefficient for load balancing was set to 1×10^{-4} , providing a weak regularization signal without dominating the primary training objective. The top- k scaling factor was set to 2.5 to moderate the effective routing logits and prevent overly sharp expert assignments. The router bias update rate was set to 1×10^{-3} during the earlier stages of training to facilitate adaptation of expert selection; however, this update rate was reduced to zero in Stage 3 to stabilize routing behavior once training shifted toward higher-quality and long-context data.

For MTP, the loss scaling factor was initially set to 0.3 and reduced to 0.1 starting from Stage 2, reflecting a transition from emphasizing auxiliary prediction signals in earlier training to prioritizing the primary language modeling objective in later stages.

3.4 Parallelism Configuration

Parallelism configurations were selected on the basis of available hardware capacity and observed communication efficiency. Pre-training was conducted on NVIDIA H200 GPUs with 140 GB memory per device. Pipeline parallelism (PP) was set to 16, while tensor parallelism was not used, as the available memory capacity was sufficient under this configuration. We employed a distributed optimizer, which partitions optimizer states across data-parallel ranks and achieves memory savings comparable to ZeRO Stage 2 (Rajbhandari et al., 2020).

Expert parallelism (EP) was fixed at 8. While higher EP degrees are feasible in principle, we did not observe proportional scaling gains beyond this point in our deployment, primarily due to network-communication characteristics rather than algorithmic constraints. In particular, our expert-parallel communication backend (DeepEP) (DeepSeek-AI, 2025) benefits from low-level access to high-performance network features (e.g., RDMA-class transports). In managed cloud environments, however, the effective bandwidth and latency can vary depending on the instance type, drivers, and networking configuration (e.g., AWS Elastic Fabric Adapter, EFA) (Amazon Web Services, 2025). Under our configuration, scaling efficiency diminished at higher EP degrees, and thus EP= 8 provided the best stability–efficiency trade-off. We note that these observations reflect our specific environment and configuration, and may differ under other hardware or networking setups.

Importantly, we incorporated these system-level effects into our end-to-end compute planning. Large-scale profiling revealed that the effective training throughput (FLOPs/s) fell short of our initial estimates under the chosen EP and networking conditions. To preserve the intended total training compute given the project timeline, we increased the training footprint from 1024 to 1536 H200 GPUs while continu-

Table 2: Per-GPU throughput measured in TFLOPs on NVIDIA H200 GPUs under a sequence length of 4,096 tokens and a global batch size of 16,384, evaluated on a 64-node cluster in our training environment. The Gain column reports the relative TFLOPs improvement introduced by each optimization compared to the previous configuration.

Optimization	Gain	TFLOPs/GPU
Baseline (EP 32, PP 8)	–	95.29
Parallelism reconfiguration (EP 8, PP 16)	+174.2%	261.27
FP8 support	+10.6%	288.97
Cross-entropy fusion	+4.4%	301.72

ing to optimize the communication and parallelization settings. This adjustment reflects a deliberate commitment to deliver a competitive model under practical constraints, backed by measurement-driven planning and efficiency-focused engineering.

Context parallelism was employed to support long-context training as the sequence length increased. Specifically, a context parallel degree of 4 was used when extending the sequence length to 16K, and a degree of 8 was applied for 32K sequences, allowing attention computation to be distributed across devices while keeping per-GPU memory usage within practical limits.

Training was initially launched on 1,024 GPUs and expanded to 1,536 GPUs after several days, with pre-training continuing for approximately 73 days in total. Following the increase in GPU count, the global batch size was scaled from 16,384 to 18,432, corresponding to an increase in tokens per step from approximately 64M to 75M.

Table 2 summarizes the impact of successive system-level optimizations on per-GPU training throughput measured in TFLOPs. The baseline configuration in this table reflects performance observed under the networking constraints of the training environment and should be interpreted as a reference point for the optimization process rather than as a general characterization of expert parallelism scalability. Subsequent improvements were achieved through parallelism reconfiguration, FP8 precision, and fused cross-entropy computation, cumulatively resulting in a substantial increase in effective training throughput.

3.5 Memory Management and Training Efficiency

Activation recomputation (Korthikanti et al., 2022) was employed to manage activation memory pressure across all training configurations, including the baseline 4K context length. Recomputation was applied selectively rather than uniformly across the model. Specifically, recomputation was enabled for the up-projection in MLA and for the MLP layers, which together accounted for the dominant portion of activation memory consumption. Other components were left unchanged to avoid unnecessary compute overhead. While full activation recomputation can further reduce peak memory usage, preliminary experiments indicated that it resulted in a substantial slowdown in training throughput, on the order of several tens of percent. Given this trade-off, selective recomputation was chosen as a practical compromise, providing sufficient memory savings while maintaining acceptable training efficiency.

FP8 precision was employed during training, corresponding to the FP8 optimization stage reported in Table 2. Specifically, the forward pass used the E4M3 format, while the backward pass used E5M2, reflecting the different numerical stability requirements of activations and gradients (Micikevicius et al., 2022). A per-tensor scaling strategy was adopted for FP8 tensors. To preserve numerical stability, the main model parameters and gradients were maintained in FP32, while the exponential moving averages and squared gradients used by the optimizer were stored in BF16. All remaining eligible tensors were represented in FP8. Empirically, the introduction of FP8 resulted in a training throughput improvement on the order of ten percent, consistent with the per-GPU TFLOPs gains shown in Table 2.

4 Post-training

A.X K1 supports user-controlled hybrid inference, where thinking and non-thinking modes are explicitly specified at inference time. Our post-training pipeline is tailored to this setting: it strengthens reasoning when thinking is enabled, while reducing mode confusion to keep non-thinking outputs concise and stable. We also apply a reinforcement learning method that remains stable and compute-efficient for large MoE models, enabling performance improvement without training instability.

Lang.	Domain	Instruct SFT		Reasoning SFT	
		Count	Ratio (%)	Count	Ratio (%)
En	Math	570k	19.6	555k	12.1
	Code	250k	8.6	625k	13.6
	Science/Knowledge	376k	12.9	1,049k	22.8
	Instruction Following	127k	4.4	88k	1.9
	Tool-use	256k	8.8	329k	7.1
	General	499k	17.2	54k	1.2
Ko	Math	198k	6.8	441k	9.6
	Code	1k	0.0	270k	5.9
	Science/Knowledge	29k	1.0	1,054k	22.9
	Instruction Following	104k	3.6	60k	1.3
	Tool-use	107k	3.7	22k	0.5
	General	389k	13.4	53k	1.2
Total		2.91M	100.0	4.60M	100.0

Table 3: Counts and ratios of data used in post-training data mixture.

4.1 Post-training Recipe

Post-training of A.X K1 comprises three stages as follows:

1. *Supervised Fine-tuning*: The initial post-training stage adopts a dual-track strategy comprising Instruct SFT and Reasoning SFT. The former is trained on general instruction datasets (e.g., summarization, rewriting, retrieval-QA, tool use) excluding the `<think>...</think>` format, while the latter focuses on reasoning-intensive tasks (e.g., math, code) incorporating `<think>...</think>` reasoning traces. Both models originate from the same pre-trained checkpoint and are fine-tuned independently.
2. *Think-Fusion SFT*: To design a model capable of selectively employing both thinking and non-thinking modes, we introduce the Think-Fusion SFT framework. This provides a compute-efficient alternative to joint multi-objective fine-tuning. Subsequently, we perform additional fine-tuning on this merged checkpoint. While the merged model inherits strong reasoning priors, it risks mode confusion, potentially over-generating verbose thoughts in non-thinking mode. We mitigate this via a Mode-Overlap Dataset (MOD) strategy: for identical prompts, the training set includes both reasoning-intensive trajectories (y_{think}) and standard direct responses ($y_{\text{non-think}}$). Fine-tuning on this mixed distribution aligns the parameter space, ensuring the model retains deep reasoning capabilities while recovering the versatility to provide concise instructions.
3. *On-policy Reinforcement Learning*: This stage further refines the model’s instruction-following behavior via on-policy reinforcement learning. The training data are drawn exclusively from the instruction-following domain, utilizing the Llama-Nemotron dataset (NVIDIA, 2025), its translated variants, and internal datasets. Methodologically, we adapt the DAPO framework (Yu et al., 2025) by integrating the GSPO loss (Zheng et al., 2025). This modification is specifically designed to enhance on-policy optimization while preserving training stability and computational efficiency.

4.2 Supervised Fine-tuning

The supervised fine-tuning (SFT) data used in A.X K1 are divided into two distinct categories based on whether the thinking mode is enabled: reasoning (thinking mode) data and instruction-following (non-thinking mode) data. Reasoning SFT is trained exclusively on thinking mode data, while Instruct SFT relies solely on non-thinking mode instruction data. When the thinking mode is enabled, the response follows the format `<think>...</think>{response}` proposed in (DeepSeek-AI, 2025; Yang et al., 2025). In contrast, when the non-thinking mode is used, the response follows the format `</think>{response}`. The domain-wise composition and proportions of the datasets used for each SFT stage are summarized in Table 3. For both stages, we minimize the standard negative log-likelihood loss over the corresponding stage-specific datasets for supervised fine-tuning.

Instruct SFT Data Construction. While the English subset primarily utilizes open-source repositories, the Korean component is constructed entirely in-house, employing data curation and augmentation to enhance both scale and quality. To ensure diverse coverage, prompts are aggregated from existing responses and newly curated instruction sets, with concise, direct responses from open-source models.

Finally, we uphold rigorous quality standards by filtering candidates through difficulty classification and correctness verification. Formally, we compile these filtered samples into a standard instruction dataset $\mathcal{D}_{\text{non-think}} = \{(x, y_{\text{non-think}})\}$.

Reasoning SFT Data Construction. The English thinking mode dataset encompasses a diverse range of open-source resources spanning mathematics, coding, STEM, and tool-use domains, alongside agentic trajectories. Crucially, it incorporates complex coding tasks requiring long-horizon planning (e.g., SWE-style debugging and code repair) as well as long-context scenarios for QA and summarization. To address the scarcity of high-quality Korean reasoning resources, we constructed a comprehensive dataset through a hybrid strategy combining proprietary data curation and strategic augmentation. This corpus integrates manually annotated domain-specific sources, such as mathematics competitions and qualification examinations, with synthetic reasoning trajectories generated via our custom pipelines involving cross-lingual translation and open-source models. This approach enables the creation of a high-density Korean reasoning corpus while preserving task diversity and structural consistency. This process yields the reasoning-intensive dataset $\mathcal{D}_{\text{think}} = \{(x, y_{\text{think}})\}$, where y_{think} contains explicit reasoning traces.

4.3 Think-Fusion SFT

Recent think-hybrid LLMs integrate thinking and non-thinking behaviors into a single checkpoint, but the fusion recipe is highly sensitive: in particular, the mixing ratio between non-reasoning and reasoning supervision varies across models (Yang et al., 2025; Bae et al., 2025) and can critically affect controllability. Consistent with these observations, prior empirical analysis (Wang et al., 2025) shows that mode separation depends on multiple interacting factors (e.g., data ratio and training schedule), often requiring iterative train-evaluate cycles to identify a workable configuration. To avoid expensive retraining sweeps over model-data mixtures, we propose a Think-Fusion SFT recipe. Specifically, we initialize the model by linearly merging the Instruction SFT model and the Reasoning SFT model using a predefined fusion coefficient. Although the linearly merged model achieves reasonable performance, the simple model merging approach can cause mode confusion, leading to degraded response quality and format instability (e.g., malformed or unclosed `<think>` tags). To this end, we fine-tune the merged model on a MODE-OVERLAP dataset, which pairs identical prompts with both reasoning-intensive and concise response trajectories. The alignment step significantly reduces format errors and facilitates flexible switching between the two modes without sacrificing performance.

Linear Model Merging Initialization. We initially construct a hybrid parameter space by linearly interpolating between a specialized reasoning model (θ_{think}) and a standard instruction-tuned model ($\theta_{\text{non-think}}$). We define the initialized weights θ_{init} as

$$\theta_{\text{init}} = \alpha \cdot \theta_{\text{think}} + (1 - \alpha) \cdot \theta_{\text{non-think}}. \quad (1)$$

Empirically, we set the mixing coefficient $\alpha = 0.8$, which was determined in collaboration with KRAFTON AI. We found that it strikes a good balance between reducing mode confusion and maintaining generation quality. This high ratio intentionally biases the model towards the reasoning distribution to ensure the retention of complex problem-solving capabilities (i.e., *thinking* features) in the latent space before the subsequent on-policy RL.

Mode-Overlap Dataset Construction. A critical challenge in training fused models is the discrepancy between the merged weights and the target output distribution. Training solely on reasoning traces ($\mathcal{D}_{\text{think}}$) can force the model to over-generate verbose thought tokens, while training only on standard data ($\mathcal{D}_{\text{non-think}}$) may degrade the merged reasoning capabilities.

To reduce mode confusion, we construct a Mode-Overlap Dataset (MOD), denoted as $\mathcal{D}_{\text{mix}}$. For a given prompt x , we curate paired responses covering both modes:

1. **Thinking Mode (y_{think}):** Responses with explicit reasoning chains, drawn from or generated to match the distribution of $\mathcal{D}_{\text{think}}$.
2. **Non-thinking Mode ($y_{\text{non-think}}$):** Direct and concise responses aligned with $\mathcal{D}_{\text{non-think}}$.

The final training objective minimizes the negative log-likelihood over the mixed distribution:

$$\mathcal{L}_{\text{SFT}} = - \sum_{(x,y) \in \mathcal{D}_{\text{mix}}} \log P(y \mid x; \theta_{\text{init}}), \quad (2)$$

where $\mathcal{D}_{\text{mix}}$ contains both (x, y_{think}) and $(x, y_{\text{non-think}})$. By supervising the merged model using the overlapping data, we encourage the model to navigate both responses, effectively preventing mode

confusion and allowing the model to utilize the strong reasoning priors injected during the merging phase while retaining conversational versatility.

4.4 On-policy Reinforcement Learning

The on-policy reinforcement learning (RL) stage is trained on unified instruction-following prompts constructed for paired thinking and non-thinking responses.

Formally, the RL dataset consists of unlabeled prompts, denoted as $\mathcal{D} = \{x_*\}$. To explicitly train both inference modes, we construct two distinct prompt variants for each base instruction I : a reasoning-inducing prompt $x_{\text{think}} = [I; \langle \text{think} \rangle]$ and a concise-response prompt $x_{\text{non-think}} = [I; \langle / \text{think} \rangle]$. These variants are treated as independent training samples within $\mathcal{D}$. This setup ensures that the optimization process reinforces the desired format and content strictly based on the control tokens appended to the prompt.

For optimization, we adopt GSPO-style sequence-level loss (Zheng et al., 2025) within the DAPO on-policy reinforcement learning framework (Yu et al., 2025). As stated in GSPO, unlike standard token-level objectives, which are sensitive to the high variance of individual token likelihoods induced by dynamic expert routing, this sequence-level formulation is particularly robust for MoE architectures.

To further ensure robustness and prevent mode confusion, we incorporate format-aware rewards:

$$R_{\text{total}}(y) = R_{\text{correct}}(y) + R_{\text{format}}(y), \quad (3)$$

where $R_{\text{correct}}(y) \in \{1, -1\}$ reflects the solution accuracy, assigning positive feedback for correctness and negative for errors. Crucially, $R_{\text{format}}(y) \in \{0, -1\}$ functions as a strict penalty term; it assigns -1 for incorrect or ambiguous usage of reasoning tokens (e.g., malformed tags) and 0 for valid structures. These choices ensure reliable user-controlled exposure of reasoning traces without compromising instruction-following quality.

5 Evaluation

5.1 Evaluation Settings

In evaluating the A.X K1 model, we adopt a benchmark selection strategy that combines (1) representative public benchmarks widely used in recent large language model evaluations and (2) internally constructed benchmarks designed to better reflect Korean-language usage and domain-specific requirements.

The evaluation is organized into seven categories: Math, Code, Knowledge, Korean, Instruction Following (IF), Long-context, and Agent; each targeting a distinct aspect of model capability. Several benchmarks are carefully designed or selected with explicit consideration of potential data contamination and over-optimization, particularly for Korean-language and instruction-following tasks.

By jointly leveraging public and in-house benchmarks, our evaluation aims to provide a balanced and reliable comparison across models, covering both globally established benchmarks and settings that are underrepresented in existing evaluations. Detailed descriptions of each benchmark are provided in Appendix B.2.

5.2 Evaluation Results of Thinking Mode

To validate the effectiveness of A.X K1, we conducted a comprehensive evaluation across diverse domains, including Knowledge, Instruction Following, Math, Code, Long Context, and Agentic capabilities. We compared our model against leading open-weight models, DeepSeek-V3.1 and GLM-4.6, as shown in Table 4.

It is important to note that A.X K1 was developed under a significantly more constrained computational budget compared to the baseline models. Despite this disparity in training resources, A.X K1 demonstrates competitive performance in several key benchmarks. Notably, in the Math domain, our model achieved a score of 89.8 on AIME25, outperforming both DeepSeek-V3.1 (88.4) and GLM-4.6 (86.0). Furthermore, in Korean-specific knowledge tasks such as KMMLU, A.X K1 recorded 80.2, proving its superior understanding of the target language context.

While the overall performance across broader benchmarks, particularly in Long Context and complex Coding tasks, shows a slight gap compared to the SOTA models, this can be primarily attributed to the difference in pre-training scale and compute resources. However, the fact that A.X K1 secures State-of-the-Art results in specific high-reasoning tasks (Math) and language-specific benchmarks (Korean) suggests that our architecture possesses strong potential for scaling.

Table 4: Comparison with large-size frontier LLMs (Thinking Mode).

Domain	Benchmark	Lang.	A.X K1 (519B-A33B)	DeepSeek-V3.1 (685B-A37B)	GLM-4.6 (357B-A32B)
Knowledge	KMMLU	Korean	80.2	76.5	79.9
	KMMLU-Redux	Korean	77.9	75.9	78.2
	KMMLU-Pro	Korean	68.1	71.4	71.8
	CLiCK	Korean	84.9	84.5	84.9
	KoBALT	Korean	48.8	59.7	59.2
	MMLU-Pro	English	81.5	85.1	82.9
	GPQA Diamond	English	74.0	77.9	78.0
Instruction Following	IFBench (prompt-loose)	English	64.7	41.5	43.4
	IFEval (prompt-strict)	English	80.4	84.4	86.1
	IFEval-ko (prompt-strict)	Korean	81.0	79.2	85.8
Math	AIME25	English	89.8	88.4	86.0
	AIME25-ko	Korean	80.4	81.3	80.4
	HRM8K	Korean	84.3	84.3	83.9
Code	LiveCodeBench v6 (25.2.1 - 25.5.1)	English	75.8	69.5	76.0
	LiveCodeBench-ko	Korean	73.1	66.2	55.9
	HumanEval+	English	87.2	86.0	83.5
	HumanEval+ ko	Korean	90.2	93.9	86.0
	MBPP+	English	93.0	99.2	98.9
	SciCode	English	32.4	39.1	38.4
Long Context	AA-LCR	English	36.0	53.3	54.3
	Humanity’s Last Exam	English	7.2	13.0	13.3
Agent	τ^2 Telecom	English	58.1	37.4	70.5

Table 5: Comparison with large-sized frontier LLMs (Non-Thinking Mode).

Domain	Benchmark	A.X K1 (519B-A33B)	DeepSeek-V3.1 (685B-A37B)	GLM-4.6 (357B-A32B)
Knowledge	KMMLU	73.0	78.7	77.7
	KMMLU-Redux	68.3	75.9	73.4
	KMMLU-Pro	60.3	67.9	68.2
	CLiCK	77.2	80.9	77.9
Instruction Following	IFBench	44.3	37.8	36.7
	IFEval	78.6	82.7	87.2
Code	HumanEval+	79.9	87.8	89.0
	HumanEval+ ko	75.6	86.6	92.1
	MBPP+	85.7	92.6	94.2

5.3 Evaluation Results of Non-Thinking Mode

In non-thinking mode, A.X K1 shows mixed performance across instruction-following, knowledge, and code benchmarks shown in Table 5. It achieves the highest score on IFBench among the compared models, while scoring lower than DeepSeek-V3.1 and GLM-4.6 on IFEval. Across knowledge benchmarks (KMMLU, KMMLU-Redux, KMMLU-Pro), A.X K1 consistently records lower scores than both comparison models, and remains slightly below DeepSeek-V3.1 on CLiCK while being comparable to GLM-4.6. In code generation benchmarks (HumanEval+, HumanEval+ ko, MBPP+), A.X K1 underperforms both DeepSeek-V3.1 and GLM-4.6, reflecting the impact of training and inference compute budgets under non-thinking inference.

5.4 Quantitative Analysis of Tokenization Efficiency

We report quantitative evaluations of the proposed tokenizer. Tokenization efficiency was measured across multiple languages and domains, including natural language, reasoning, mathematics, and code, using average token count as the primary metric. The tokenizer consistently reduced token counts across languages, achieving particularly strong efficiency in Korean. Improvements were also observed in English and code, demonstrating efficiency competitive with English SOTA models. Table 6 reports the average token counts across languages and domains.

Table 6: Tokenization Efficiency Comparison (Average Tokens per Sample). **Bold** denotes the best efficiency, and Underline indicates the second-best ones.

Dataset	A.X K1	OpenAI o200k	Qwen3
Korean (General)	526.72	<u>811.71</u>	905.82
Korean (Reasoning)	5520.06	<u>7273.03</u>	7890.16
Korean (Math)	812.99	<u>1173.98</u>	1297.56
English (General)	<u>792.28</u>	769.31	794.79
English (Reasoning)	<u>6052.71</u>	5872.95	6192.77
Chinese	<u>1050.24</u>	1069.66	914.05
Japanese	871.30	1040.50	<u>935.87</u>
Spanish	<u>1059.73</u>	937.10	1102.41
Code	<u>1693.33</u>	1680.74	1694.26

6 Limitations

While A.X K1 demonstrates competitive performance and validates our efficient training and post-training recipe, several limitations remain, largely driven by real-world constraints and stability-focused engineering trade-offs. These limitations also delineate a clear roadmap for future iterations.

- **Infrastructure Constraints on Scaling:** Our training environment imposed system-level constraints on parallelism. In particular, we capped Expert Parallelism (EP) at 8 because scaling efficiency diminished beyond this point under our networking configuration, reflecting communication characteristics rather than algorithmic limits. This restricted our ability to explore finer-grained expert partitioning and higher EP degrees that may yield additional capacity. Future work will benefit from improved interconnect characteristics and tighter integration between expert-parallel communication and the underlying network stack.
- **Optimization under Resource Constraints:** To maximize attainable performance under fixed time and GPU budget constraints, we selected model configurations guided by theoretical scaling laws. Despite parameter-level optimization, these constraints led to modest performance shortfalls relative to similarly sized models trained with larger compute budgets. We expect that increasing available compute will allow us to relax these constraints and further improve model performance.
- **Scope of Modalities:** A.X K1 is currently text-only. While it exhibits strong reasoning performance, it lacks native multimodal understanding. We plan to extend the model with native multimodal capabilities in future works.

7 Conclusion

In this work, we presented A.X K1, a 519B-parameter MoE language model that bridges high-capacity reasoning with practical inference efficiency. Under fixed and time-bounded resources, we followed vocabulary and MoE scaling principles and used FLOP-based budgeting to select a principled trade-off between model scale and training tokens, demonstrating that careful system-aware design can yield globally competitive capability. We also introduced *Think-Fusion*, a post-training recipe that combines dual-track SFT with GSPO-style on-policy reinforcement learning to enable explicit, user-controllable switching between *thinking* and *standard* modes within a single unified checkpoint. Across mathematics, coding, and general-knowledge benchmarks in both English and Korean, our evaluations show that A.X K1 is competitive with strong open-source baselines.

Beyond the theoretically optimal architecture defined by scaling laws, this project focuses on the system-level engineering required to translate that design into effective large-scale MoE training under real-world infrastructure constraints. Empirical profiling revealed systematic gaps between theoretical and realized throughput, driven by communication overhead, load imbalance, and execution-level inefficiencies. To preserve the intended training compute and schedule, we expanded the cluster from 1,024 to 1,536 H200 GPUs and iteratively optimized stability, utilization, and throughput. This approach enabled stable convergence and predictable training behavior within fixed resource and timeline constraints.

Looking ahead, we will prioritize native multimodal capabilities and continued scaling toward the trillion-parameter regime. Building on the system and optimization insights established here, we aim to further advance both model capability and the efficiency of large-scale training and inference.

A Contributors

All authors are listed alphabetically by last name, then first name. Names marked with an asterisk (*) indicate authors whose affiliations differ from their affiliation at the time of this work.

Core Contributors

Sung Jun Cheon, Jaekyung Cho*, Seongho Choi, Hyunjun Eun, Seokhwan Jo, Jaehyun Jun*, Minsoo Kang, Jin Kim, Jiwon Kim, Minsang Kim, Sungwan Kim, Seungsik Kim, Tae Yoon Kim, Youngrang Kim, Hyeongmun Lee, Sangyeol Lee, Sungeun Lee, Youngsoon Lee, Yujin Lee, Seongmin Ok, Chanyong Park*, Hyewoong Park*, Junyoung Park, Hyunho Yang, Subin Yi

Contributors

Soohyun Bae, Dhammiko Arya, Yongseok Choi, Sangho Choi, Dongyeon Cho, Seungmo Cho, Gyoungun Han, Yong-jin Han, Seokyoung Hong, Hyeon Hwang*, Wonbeom Jang, Minjeong Ju, Wonjin Jung, Keummin Ka*, Sungil Kang, Dongnam Kim, Joonghoon Kim, Jonghwi Kim*, SaeRom Kim, Sangjin Kim, Seongwon Kim, Youngjin Kim, Seojin Lee, Sunwoo Lee, Taehoon Lee, Chanwoo Park*, Sohee Park, Sooyeon Park, Yohan Ra, Sereimony Sek, Seungyeon Seo, Gun Song, Sanghoon Woo*, Janghan Yoon*, Sungbin Yoon

Acknowledgement

This work was supported by the Ministry of Science and ICT(MSIT), Republic of Korea, through the National IT Industry Promotion Agency(NIPA) (Grant No. PJT-25-080042). This research was conducted as part of the Sovereign AI Foundation Model Project(Data Track), organized by the Ministry of Science and ICT(MSIT) and supported by the National Information Society Agency(NIA), S.Korea. (Grant No. 2025-AIData-WII43). We acknowledge [KRAFTON AI](#)—in particular, Gyeongman Kim, Junhyuck Kim, Haechan Kim, and Beongjun Choi—for their collaboration in the development of the Think-Fusion SFT recipe.

B Appendix

B.1 Pre-train Data Category Mixture

Table 7: Dataset Composition by Category Across Training Stages.

Category	Stage 1 (%)	Stage 2 (%)	Stage 3 (%)
Web	53.41	39.79	33.32
Code	17.13	29.42	25.50
Encyclopedia	14.84	0.64	0.72
Q&A	13.04	8.95	5.38
Books	0.77	4.35	3.70
Academic Literature	0.66	7.23	3.87
Mathematics	–	5.41	9.24
STEM	–	2.47	9.07
Reasoning	–	0.82	8.32
Others	0.16	0.91	0.87
Total	100.00	100.00	100.00

B.2 Details of Evaluation Benchmarks

Math For English benchmarks, we use AIME25 (Zhang & Math-AI, 2025), a prestigious high-school-level mathematics competition problem set drawn from the 2025 American Invitational Mathematics Examination (AIME). Korean math performance is evaluated using AIME25-ko, a Korean translation of AIME25, and HRM8K (Ko et al., 2025). All math benchmarks are evaluated based on the correctness of the final numerical answer, using a chat-based chain-of-thought (Chat CoT) prompting setup that encourages sufficient intermediate reasoning. We evaluate AIME25 using a maximum context length of 128K tokens. The evaluation prompt and usage examples for AIME25 and HRM8K are shown in Fig. 3 and Fig. 4, respectively.

Evaluation Prompt for AIME25

Solve the following math problem efficiently and clearly. The last line of your response should be of the following format: 'Therefore, the final answer is: $\boxed{\text{ANSWER}}$. I hope it is correct' (without quotes) where ANSWER is just the final number or expression that solves the problem. Think step by step before answering.

Find the sum of all integer bases $b > 9$ for which 17_b is a divisor of 97_b .

Figure 3: One example of evaluation prompt for AIME25. The model’s response is evaluated by comparing the ANSWER field against the ground-truth answer.

Evaluation Prompt for HRM8K

다음 수학 문제를 효율적이고 명확하게 풀어주세요. 마지막 줄의 답변은 다음 형식이어야 합니다: '따라서 최종 답은: $\boxed{\text{ANSWER}}$. 맞기를 바랍니다.' (따옴표 없이, ANSWER는 문제를 해결한 최종 숫자나 표현식입니다). 문제를 풀기 전에 한 단계씩 생각해주세요.

Janet의 오리는 하루에 16개의 알을 낳습니다. 그녀는 매일 아침으로 3개를 먹고, 친구들을 위해 머핀을 구울 때 4개를 사용합니다. 남은 계란은 매일 농산물 시장에서 신선한 오리 알 하나당 2달러에 판매합니다. 그녀는 매일 농산물 시장에서 얼마를 버나요?

Figure 4: One example of evaluation prompt for HRM8K. The model’s response is evaluated by comparing the ANSWER field against the ground-truth answer.

Code We use HumanEval+, HumanEval+ko, MBPP+, LiveCodeBench (Jain et al., 2024), and LiveCodeBench-ko. Generated code is executed using a Python interpreter, and correctness is determined by matching execution results with expected outputs. The LiveCodeBench series includes recently updated problems, reducing the risk of data contamination; we evaluate only the February, March, and April 2025 subsets. All LiveCodeBench evaluations are conducted with a context length of 128K tokens.

Knowledge For English knowledge benchmarks, we use MMLU (Hendrycks et al., 2021), MMLU-Pro (Wang et al., 2024), GPQA Diamond (Rein et al., 2024), and Humanity’s Last Exam (Phan et al., 2025). MMLU assesses undergraduate-level knowledge across diverse academic fields, while MMLU-Pro increases difficulty by expanding the number of answer choices. GPQA Diamond consists of high-difficulty, graduate-level questions in physics, chemistry, and biology, and HLE evaluates advanced knowledge and reasoning capabilities. MMLU and MMLU-Pro are evaluated using the chat-based chain-of-thought (Chat CoT) prompting. The evaluation prompts and usage examples for MMLU and MMLU-Pro are shown in Figures 5 and 6, respectively.

Evaluation Prompt for MMLU

Given the following question and four candidate answers (A, B, C and D), choose the best answer.

Question: What is the greatest common divisor of ?

A. 1
B. 2049
C. 2048
D. 2047

- For simple problems:
Directly provide the answer with minimal explanation.

- For complex problems:
Use this step-by-step format:
Step 1: [Concise description]
[Brief explanation]
Step 2: [Concise description]
[Brief explanation]

Regardless of the approach, always conclude with:
The best answer is [the.answer.letter].
where the [the.answer.letter] is one of A, B, C or D.

Let’s think step by step.

Figure 5: One example of an evaluation prompt for MMLU. The model’s response is evaluated by comparing the [the.answer.letter] field against the ground-truth answer.

Evaluation Prompt for MMLU-Pro

Answer the following multiple-choice question. The last line of your response should be in the following format: ‘Answer: A/B/C/D’ (e.g. ‘Answer: A’).

Functions of the law include all but which of the following?

A) defining the limits of government power
B) regulating the use of public spaces
C) keeping the peace
D) maximizing individual freedom
E) maintaining order and stability
F) preventing environmental degradation
G) providing a basis for compromise
H) promoting social justice
I) promoting the principles of the free enterprise system
J) encouraging economic growth

Figure 6: An example MMLU-Pro evaluation prompt. Model outputs are scored by parsing the final Answer: field and matching the selected option with the ground-truth answer.

Korean For Korean knowledge evaluation, we use KMMLU (Son et al., 2024), KMMLU-Redux (Hong et al., 2025), KMMLU-Pro (Hong et al., 2025), CLiCK (Kim et al., 2024), and KoBALT (Shin et al., 2025). KMMLU is the Korean counterpart of MMLU, while KMMLU-Redux corrects and refines errors in the original KMMLU. KMMLU-Pro includes questions based on the Korean national professional qualification exams. CLiCK and KoBALT evaluate Korean knowledge and reasoning. For KMMLU, we adopt

the chat-based chain-of-thought (Chat CoT) evaluation protocol proposed in (Dubey et al., 2024). In contrast, CLiCK is evaluated in a standard zero-shot setting, consistent with the evaluation protocol used for pre-trained models.

Instruction Following This category evaluates the model’s ability to follow given instructions accurately. We use IFEval (Zhou et al., 2023), IFEval-ko, and IFBench (Pyatkin et al., 2025), which assess compliance with explicit constraints such as output format, length limits, required expressions, or language usage. IFEval is evaluated using predefined Python scripts, while IFEval-ko is an in-house Korean translation of IFEval. All benchmarks are evaluated using rule-based criteria with predefined evaluation protocols.

Long-context Long-context understanding is evaluated using the AA-LCR (Artificial Analysis Team, 2025) benchmark. This benchmark evaluates the model’s ability to maintain coherence and retain key information when generating responses based on long input contexts. All evaluations are conducted with a maximum context length of 128K tokens, reflecting the model’s long-context capability.

Agent τ^2 -Bench Telecom (Barres et al., 2025) evaluates agent performance in the Telecom domain, such as customer service interactions.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Amazon Web Services. Elastic fabric adapter for ai/ml and hpc workloads on amazon ec2. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html>, 2025. AWS documentation. Accessed: 2026-01-06.
- Artificial Analysis Team. Artificial analysis long context reasoning benchmark(lcr), 2025.
- Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. Morphbpe: A morpho-aware tokenizer bridging linguistic complexity for efficient llm training across morphologies, 2025. URL <https://arxiv.org/abs/2502.00894>.
- Kyunghoon Bae, Eunbi Choi, Kibong Choi, Stanley Jungkyu Choi, Yemuk Choi, Kyubeen Han, Seokhee Hong, Junwon Hwang, Taewan Hwang, Joonwon Jang, Hyojin Jeon, Kijeong Jeon, Gerrard Jeongwon Jo, Hyunjik Jo, Jiyeon Jung, Euisoon Kim, Hyosang Kim, Jihoon Kim, Joonkee Kim, Seonghwan Kim, Soyeon Kim, Sunkyoung Kim, Yireun Kim, Yongil Kim, Youchul Kim, Edward Hwayoung Lee, Gwangho Lee, Haeju Lee, Honglak Lee, Jinsik Lee, Kyungmin Lee, Sangha Park, Young Min Paik, Yongmin Park, Youngyong Park, Sanghyun Seo, Sihoon Yang, Heuiyeen Yeen, Sihyuk Yi, and Hyeongu Yun. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes, 2025. URL <https://arxiv.org/abs/2507.11407>.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pp. 41–48, 2009.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- DeepSeek-AI. DeepEP: An efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025. GitHub repository. Accessed: 2026-01-06. (Optionally record the commit or release tag used in experiments.).
- DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- GLM-4.5 Team. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models, 2025. URL <https://arxiv.org/abs/2508.06471>.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024. URL <https://arxiv.org/abs/2404.19737>.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 30016–30030, 2022.

-
- Seokhee Hong, Sunkyoung Kim, Guijin Son, Soyeon Kim, Yeonjung Hong, and Jinsik Lee. From kmmlu-redux to kmmlu-pro: A professional korean benchmark suite for llm evaluation, 2025. URL <https://arxiv.org/abs/2507.08924>.
- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Eunsu Kim, Juyoung Suk, Philhoon Oh, Haneul Yoo, James Thorne, and Alice Oh. CLiCK: A benchmark dataset of cultural and linguistic intelligence in korean, 2024.
- Kimi Team. Kimi K2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Hyunwoo Ko, Guijin Son, and Dasol Choi. Understand, solve and translate: Bridging the multilingual mathematical reasoning gap. *arXiv preprint arXiv:2501.02448*, 2025.
- Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models, 2022. URL <https://arxiv.org/abs/2205.05198>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 formats for deep learning, 2022. URL <https://arxiv.org/abs/2209.05433>.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20286–20332, 2025.
- NVIDIA. Llama-Nemotron: Efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.00949>.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- Perplexity AI. Perplexity.ai. <https://www.perplexity.ai/>. AI search engine with live web retrieval and explicit source citations.
- Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following, 2025.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020. URL <https://arxiv.org/abs/1910.02054>.

-
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pp. 1715–1725, 2016.
- Hyopil Shin, Sangah Lee, Dongjun Jang, Wooseok Song, Jaeyoon Kim, Chaeyoung Oh, Hyemi Jo, Youngchae Ahn, Sihyun Oh, Hyohyeong Chang, Sunkyoung Kim, and Jinsik Lee. KoBALT: Korean benchmark for advanced linguistic tasks, 2025. URL <https://arxiv.org/abs/2505.16125>.
- SKT. A.X 4.0, 2025a. URL <https://huggingface.co/skt/A.X-4.0>.
- SKT. A.X 4.0 Light, 2025b. URL <https://huggingface.co/skt/A.X-4.0-Light>.
- SKT. A.X 4.0 VL Light, 2025c. URL <https://huggingface.co/skt/A.X-4.0-VL-Light>.
- SKT. A.X Encoder-base, 2025d. URL <https://huggingface.co/skt/A.X-Encoder-base>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Guijin Son, Hanwool Lee, Sungdong Kim, Seungone Kim, Niklas Muennighoff, Taekyoon Choi, Cheonbok Park, Kang Min Yoo, and Stella Biderman. KMMLU: Measuring massive multitask language understanding in korean. *arXiv preprint arXiv:2402.11548*, 2024.
- Chaofan Tao, Qian Liu, Longxu Dou, Niklas Muennighoff, Zhongwei Wan, Ping Luo, Min Lin, and Ngai Wong. Scaling laws with vocabulary: Larger models deserve larger vocabularies. *Advances in Neural Information Processing Systems*, 37:114147–114179, 2024.
- Changxin Tian, Kunlong Chen, Jia Liu, Ziqi Liu, Zhiqiang Zhang, and Jun Zhou. Towards greater leverage: Scaling laws for efficient mixture-of-experts language models, 2025. URL <https://arxiv.org/abs/2507.17702>.
- Shouren Wang, Wang Yang, Xianxuan Long, Qifan Wang, Vipin Chaudhary, and Xiaotian Han. Demystifying hybrid thinking: Can llms truly switch between think and no-think?, 2025. URL <https://arxiv.org/abs/2510.12680>.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Haoyi Xiong, Jiang Bian, Yuchen Li, Xuhong Li, Mengnan Du, Shuaiqiang Wang, Dawei Yin, and Sumi Helal. When search engine services meet large language models: Visions and challenges. *arXiv preprint arXiv:2407.00128*, 2024. URL <https://arxiv.org/abs/2407.00128>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019. URL <https://arxiv.org/abs/1910.07467>.
- Yifan Zhang and Team Math-AI. American invitational mathematics examination (aime) 2025, 2025.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization, 2025. URL <https://arxiv.org/abs/2507.18071>.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.